

# ブラウザ自動化基盤(Playwright) - 詳細説明書

## このドキュメントについて

このドキュメントは、Milestone 2の基盤3「ブラウザ自動化基盤(Playwright)」の詳細な技術説明書です。Playwrightを使用した200アカウントの自動化、コンテキスト分離、並列実行、エラー復旧について包括的に説明しています。

対象読者: 開発者、自動化エンジニア、技術リード

難易度: 上級

推定読了時間: 25-30分

## 概要

Milestone 2の第3の基盤として、200アカウントのブラウザ自動化を実現するPlaywright基盤を構築しました。このシステムは、1アカウント=1コンテキストの完全分離、自動クラッシュ復旧、手動操作検知、安全な並列実行を提供します。

## 技術スタック

- ブラウザ自動化: Playwright (Chromium)
- バックエンド: Python + Flask
- 並列実行: ThreadPoolExecutor
- 暗号化: Cryptography (Fernet)
- データベース: Supabase (PostgreSQL Online)

## システムアーキテクチャ

### 全体構成図



## 核心原則

### 1. 1アカウント = 1 Browser Context

なぜ重要か

✕ 悪い実装: 1ブラウザ = 200アカウント

```
browser = playwright.chromium.launch()
page = browser.new_page()

for account in accounts:
    login(page, account) # 同じページを使い回し
    do_work(page, account)
    logout(page, account)
```

問題:

- Cookieが混在
- localStorage が共有
- セッション競合
- ログアウト漏れで次のアカウントに影響

✓ 正しい実装: 1コンテキスト = 1アカウント

```
browser = playwright.chromium.launch() # 1つのブラウザ

for account in accounts:
    context = browser.new_context() # 新しいコンテキスト
    page = context.new_page()
    login(page, account)
    do_work(page, account)
    context.close() # コンテキストを閉じる
```

利点:

- Cookieが完全分離
- localStorage が独立
- セッション競合なし
- アカウント間の影響なし

## 実装コード

```
class BrowserManager:
    def start(self):
        """1つのブラウザインスタンスを起動"""
        self.playwright = sync_playwright().start()
        self.browser = self.playwright.chromium.launch(
            headless=True,
            args=[
                '--disable-blink-features=AutomationControlled',
                '--disable-dev-shm-usage',
                '--no-sandbox'
            ]
        )

    @contextmanager
    def create_context(self, account_id: str):
        """アカウント専用のコンテキストを作成"""
        # 完全分離されたコンテキスト
        context = self.browser.new_context(
            viewport={'width': 1920, 'height': 1080},
            user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
        )

        try:
            yield context
        finally:
            context.close() # 自動クリーンアップ
```

## コンテキスト分離の視覚化

### Browserインスタンス (共有)

Context 1: user001  
└ Cookie: session\_id=abc123  
└ localStorage: token=xyz789  
└ Page: https://doors1.shinchakun.info/

Context 2: user002  
└ Cookie: session\_id=def456 ← 独立  
└ localStorage: token=uvw012 ← 独立  
└ Page: https://doors2.shinchakun.info/

Context 3: user003  
└ Cookie: session\_id=ghi789 ← 独立  
└ localStorage: token=rst345 ← 独立  
└ Page: https://doors1.shinchakun.info/

## 2. URL自動判別とキャッシュ

### URL判別フロー

#### 1. キャッシュ確認

```
cached_url = account.login_url  
if cached_url:  
    return cached_url # 高速(~1秒)
```

↓ キャッシュなし

#### 2. URL自動判別

```
for url in settings.LOGIN_URLS:  
    page.goto(url, timeout=5000)  
    password_input = page.locator("input[type='password']")  
    if password_input.is_visible():  
        return url # 発見
```

↓

#### 3. URLをキャッシュ

```
account.login_url = url  
database.save(account)
```

↓

次回以降は高速

初回: ~5秒 (全URL試行)

2回目以降: ~1秒 (キャッシュ使用)

### 実装コード

```
def resolve_base_url(page: Page, account_id: str) -> str:
    """
    ログインURLを自動判別

    試行順序:
    1. https://doors1.shinchakun.info/dokodemo/#/
    2. https://doors2.shinchakun.info/dokodemo/#/

    判別方法:
    - ページにアクセス
    - input[type='password'] が表示されるか確認
    - 表示されればそのURLを返す
    """
    for url in settings.LOGIN_URLS:
        try:
            page.goto(url, timeout=5000, wait_until="domcontentloaded")

            password_input = page.locator("input[type='password']")
            if password_input.is_visible(timeout=2000):
                return url # 有効なURLを発見

        except:
            continue # 次のURLを試行

    raise AutomationError("Login URL not found")

def get_cached_url(account) -> Optional[str]:
    """データベースからキャッシュされたURLを取得"""
    if account and account.login_url:
        return account.login_url
    return None
```

### 3. 自動ログイン処理

#### ログインフロー

##### 1. パスワード復号化

```
password = decrypt_password(account.password_encrypted)
```

↓

##### 2. ログインフォーム入力

```
page.locator("input[name='login_id']").fill(username)
page.locator("input[type='password']").fill(password)
```

↓

##### 3. 送信ボタンクリック

```
page.locator("button[type='submit']").click()
```

↓

##### 4. ナビゲーション待機

```
page.wait_for_load_state("networkidle", timeout=15000)
```

↓

##### 5. トークン取得

```
token = page.evaluate("() => localStorage.getItem('token')")
if token:
    return token # 成功
```

↓

##### 6. URL確認(代替確認)

```
if "login" not in page.url and "dokodemo" in page.url:
    return "" # トークンなしでも成功とみなす
```

## 実装の特徴

```
def login(page: Page, account: Account) -> str:
    """ログイン処理"""
    # パスワード復号化
    password = decrypt_password(account.password_encrypted)

    # フォーム入力
    page.locator("input[name='login_id']").fill(account.username)
    page.locator("input[type='password']").fill(password)

    # 送信ボタン(複数セレクトで試行)
    submit_button = page.locator("button[type='submit']")
    if not submit_button.is_visible(timeout=5000):
        # 代替セレクト
        submit_button = page.locator("button:has-text('ログイン')")
        if not submit_button.is_visible(timeout=2000):
            submit_button = page.locator("input[type='submit']")

    submit_button.click()

    # ナビゲーション待機
    page.wait_for_load_state("networkidle", timeout=15000)

    # トークン取得
    token = page.evaluate("() => localStorage.getItem('token')")
    if not token:
        # トークンなしでもログイン成功の可能性
        if "login" not in page.url.lower() and "dokodemo" in page.url:
            return "" # 成功
        raise LoginError("Login failed")

    return token
```

## 4. 手動操作検知

### 検知原理

#### 手動操作検知の2つの方法

##### 1. トークン監視

初期化: `last_token = localStorage.getItem('token')`

チェック時:

`current_token = localStorage.getItem('token')`

if `current_token != last_token`:

# 手動ログインが発生した

raise `ManualOperationDetected()`

##### 2. DOMフィンガープリント

初期化: `last_dom = hash(body.innerHTML[:1500])`

チェック時:

`current_dom = hash(body.innerHTML[:1500])`

if `current_dom != last_dom`:

# 手動ナビゲーションが発生した

raise `ManualOperationDetected()`

### 実装コード

```

class ManualOperationDetector:
    def __init__(self, account_id: str):
        self.account_id = account_id
        self.last_token = None
        self.last_dom_fingerprint = None

    def initialize(self, page: Page):
        """初期状態を記録"""
        self.last_token = self._get_token(page)
        self.last_dom_fingerprint = self._get_dom_fingerprint(page)

    def check(self, page: Page):
        """手動操作をチェック"""
        current_token = self._get_token(page)
        current_dom = self._get_dom_fingerprint(page)

        token_changed = (current_token != self.last_token and
                          self.last_token is not None)

        dom_changed = (current_dom != self.last_dom_fingerprint and
                       self.last_dom_fingerprint is not None)

        if token_changed or dom_changed:
            raise ManualOperationDetectedError(
                f"Manual operation detected: "
                f"token_changed={token_changed}, dom_changed={dom_changed}"
            )

    def _get_token(self, page: Page) -> Optional[str]:
        """トークン取得"""
        return page.evaluate("() => localStorage.getItem('token')")

    def _get_dom_fingerprint(self, page: Page) -> Optional[str]:
        """DOMフィンガープリント取得(最初の1500文字)"""
        return page.evaluate("""
            () => {
                const html = document.body.innerHTML.slice(0, 1500);
                return btoa(unescape(encodeURIComponent(html)));
            }
        """)

```

## 検知タイミング

| 自動化処理フロー                                     |  |
|----------------------------------------------|--|
| 1. ログイン後                                     |  |
| manual_detector.initialize(page)   # 初期状態を記録 |  |
| 2. 業務ロジック実行前                                 |  |
| manual_detector.check(page)   # 手動操作チェック     |  |
| ↓                                            |  |
| if 手動操作検知:                                   |  |
| account.status = MANUAL_OPERATION            |  |
| return   # スキップ                              |  |
| 3. 各操作後(オプション)                               |  |
| manual_detector.check(page)   # 再チェック        |  |
| manual_detector.update_state(page)   # 状態更新  |  |

## 5. クラッシュ検知と自動復旧

### クラッシュ検知

```
def detect_app_crash(page: Page) -> bool:
    """
    アプリクラッシュ(赤い×)を検知

    複数のセレクタで試行:
    - .red-x
    - [class*='red-x']
    - [class*='error-x']
    - div:has-text('×')
    """
    red_x_selectors = [
        ".red-x",
        "[class*='red-x']",
        "[class*='error-x']",
        "[class*='crash']",
        "div:has-text('×')",
        ".app-crash"
    ]

    for selector in red_x_selectors:
        try:
            if page.locator(selector).is_visible(timeout=1000):
                return True # クラッシュ検知
        except:
            continue

    return False # クラッシュなし
```

## 復旧フロー



## 実装コード

```

def handle_app_crash(
    page, context, account_id, browser_manager, account
) -> Page:
    """アプリクラッシュを復旧"""
    logger.critical(f"[{account_id}] App crash detected, restarting...")

    # 1. 古いコンテキストを閉じる
    context.close()

    # 2. 新しいコンテキストを作成
    new_context = browser_manager.browser.new_context(
        viewport={'width': 1920, 'height': 1080},
        user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
    )
    new_context.set_default_timeout(10000)
    new_context.set_default_navigation_timeout(30000)

    # 3. 新しいページを作成
    new_page = new_context.new_page()

    logger.info(f"[{account_id}] App restarted successfully")
    return new_page

```

## 6. Worker関数(完全なワークフロー)

### 全体フロー

```
run_account_job(browser_manager, account, task_type)
```

#### 1. ステータス更新

```
account.status = PROCESSING
```

#### 2. ブラウザコンテキスト作成

```
with browser_manager.create_context(account_id) as ctx:  
    page = ctx.new_page()
```

#### 3. URL解決

```
base_url = get_cached_url(account)  
if not base_url:  
    base_url = resolve_base_url(page, account_id)  
account.login_url = base_url # キャッシュ
```

#### 4. ログイン

```
with log_manager.log_operation(account_id, LOGIN):  
    token = login(page, account)  
    account.session_token = token
```

#### 5. 手動操作検知器初期化

```
manual_detector = ManualOperationDetector(account_id)  
manual_detector.initialize(page)
```

#### 6. クラッシュチェック

```
if detect_app_crash(page):  
    page = handle_app_crash(...)  
    login(page, account) # 再ログイン
```

#### 7. 手動操作チェック

```
manual_detector.check(page)  
# 手動操作検知時はスキップ
```

#### 8. 業務ロジック実行

```
if task_type == "schedule_update":  
    execute_schedule_update(page, account)  
elif task_type == "wait_reception":  
    execute_wait_reception(page, account)
```

#### 9. 成功処理

```
account.last_success_at = now()  
account.status = IDLE  
account.error_count = 0
```

実装コード(抜粋)

```

def run_account_job(
    browser_manager: BrowserManager,
    account: Account,
    task_type: str = "schedule_update"
) -> Dict[str, Any]:
    """アカウント処理Worker関数"""
    account_id = str(account.id)

    try:
        # ブラウザコンテキスト作成(1アカウント = 1 Context)
        with browser_manager.create_context(account_id) as context:
            page = context.new_page()

            # URL解決
            base_url = get_cached_url(account) or resolve_base_url(page, account_id)

            # ログイン
            with log_manager.log_operation(account_id, ActionType.LOGIN) as login_ctx:
                token = login(page, account)
                login_ctx.log_success("Login successful")

            # 手動操作検知器初期化
            manual_detector = ManualOperationDetector(account_id)
            manual_detector.initialize(page)

            # クラッシュチェック
            if detect_app_crash(page):
                page = handle_app_crash(page, context, account_id, browser_manager, account)
                login(page, account) # 再ログイン
                manual_detector.initialize(page)

            # 手動操作チェック
            manual_detector.check(page)

            # 業務ロジック実行
            if task_type == "schedule_update":
                execute_schedule_update(page, account, manual_detector, log_ctx)
            elif task_type == "wait_reception":
                execute_wait_reception(page, account, manual_detector, log_ctx)

            # 成功処理
            account_repo.update_last_success(account_id)
            account_repo.update_status(account_id, AccountStatus.IDLE)

            return {'success': True}

    except ManualOperationDetectedError:
        # 手動操作検知時はスキップ
        account_repo.update_status(account_id, AccountStatus.MANUAL_OPERATION)
        return {'success': False, 'error': 'Manual operation detected'}

    except Exception as e:
        # エラー処理
        account_repo.increment_error_count(account_id, str(e))
        return {'success': False, 'error': str(e)}

```

## 7. 並列実行エンジン

### 並列実行の原則

### 並列実行の安全な上限

- ✕ 200並列は禁止
  - リソース枯渇
  - ネットワーク過負荷
  - サーバー側の制限
- ☑ 5~10並列が現実的
  - 安定性
  - パフォーマンス
  - エラー分離

### 実装構造

```
class ParallelExecutor:
    def __init__(self, max_workers=10):
        # 安全のため10に制限
        self.max_workers = min(max_workers, 10)

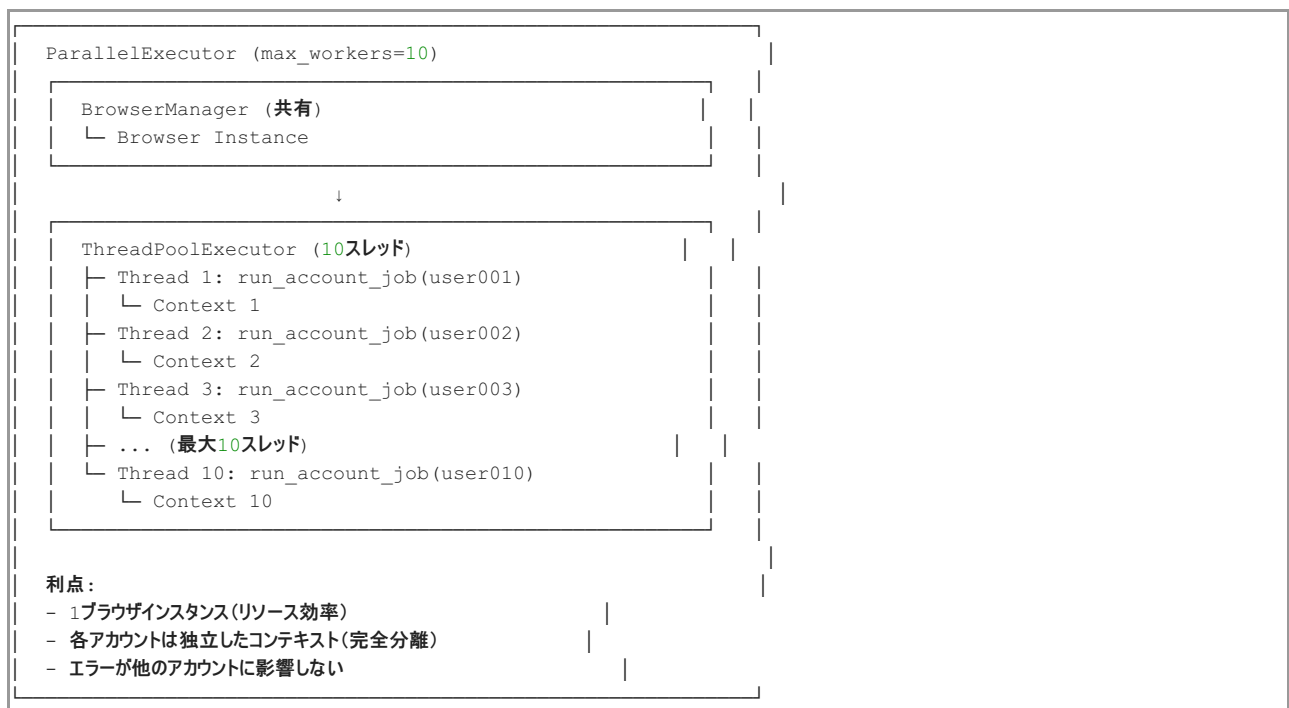
    def execute_accounts(self, accounts, task_type):
        """複数アカウントを並列実行"""
        # 1. ブラウザマネージャー起動(共有)
        browser_manager = BrowserManager()
        browser_manager.start()

        try:
            # 2. 並列実行
            with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
                # タスクを送信
                futures = {
                    executor.submit(run_account_job, browser_manager, account, task_type): account
                    for account in accounts
                }

                # 完了を待機
                for future in as_completed(futures):
                    account = futures[future]
                    try:
                        result = future.result()
                        # 結果を集計
                    except Exception as e:
                        # エラーハンドリング
                        pass

        finally:
            # 3. ブラウザマネージャー停止
            browser_manager.stop()
```

### 並列実行の視覚化



## パフォーマンス最適化

### リソース使用量

| リソース使用量の比較                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>✕ 200ブラウザインスタンス</div><div><div>- メモリ: <math>200 \times 500\text{MB} = 100\text{GB}</math></div><div>- CPU: <math>200 \times 5\% = 1000\%</math></div><div>- 現実的でない</div></div></div>                      |
| <div><div>☑ 1ブラウザ + 10コンテキスト</div><div><div>- メモリ: <math>500\text{MB} + (10 \times 100\text{MB}) = 1.5\text{GB}</math></div><div>- CPU: <math>5\% + (10 \times 2\%) = 25\%</math></div><div>- 実用的</div></div></div> |

### タイムアウト設定

```
# コンテキスト作成時
context.set_default_timeout(10000) # 10秒
context.set_default_navigation_timeout(30000) # 30秒

# ページ操作
page.locator("button").click(timeout=5000)
page.wait_for_load_state("networkidle", timeout=15000)
```

### エラー耐性

```
# リトライ機能(1回まで)
try:
    page.click("button#save")
except TimeoutError:
    logger.warning("First attempt failed, retrying...")
    page.reload()
    page.click("button#save")
```

## まとめ

## 実装した機能

- ✓ 完全分離: 1アカウント = 1 Browser Context
- ✓ URL自動判別: 複数URLから自動選択 + キャッシュ
- ✓ 自動ログイン: パスワード復号化 + フォーム自動入力
- ✓ 手動操作検知: トークン監視 + DOMフィンガープリント
- ✓ クラッシュ復旧: 自動検知 + 自動再起動
- ✓ 並列実行: 安全な並列実行(最大10ワーカー)
- ✓ エラー分離: 1アカウントの失敗が他に影響しない

## 技術スタック

- ブラウザ自動化: Playwright (Chromium)
- 並列実行: ThreadPoolExecutor
- コンテキスト管理: Python Context Manager
- 暗号化: Cryptography (Fernet)

## 次のステップ

この基盤の上に、以下の業務ロジックが実装されます:

- スケジュール更新処理
- 待機接客配信制御
- ビジネスルールの実装

---

## 参考資料

### 実装ファイル

- **BrowserManager**: backend/src/automation/browser\_manager.py
- **Worker関数**: backend/src/automation/worker.py
- **URL Resolver**: backend/src/automation/url\_resolver.py
- **Login Handler**: backend/src/automation/login\_handler.py
- **Manual Detector**: backend/src/automation/manual\_detector.py
- **Crash Detector**: backend/src/automation/crash\_detector.py
- **Parallel Executor**: backend/src/automation/executor.py

### API統合

- **API Routes**: backend/src/web/api/routes.py
  - POST /api/automation/execute - 全アカウント実行
  - POST /api/automation/execute/<account\_id> - 単一アカウント実行
  - POST /api/automation/test - テスト実行

### 関連ドキュメント

- **アカウント管理データベース**: docs/milestone2/account\_database\_detailed\_jp.md
- **ログ基盤**: docs/milestone2/logging\_infrastructure\_detailed\_jp.md
- **競争対策モジュール**: docs/milestone2/competitive\_countermeasure\_detailed\_jp.md

---

ドキュメントバージョン: 2.0

最終更新: 2025年12月26日

実装状況: 完全実装完了

技術スタック: Playwright + Python + TypeScript